AI-ASSISTED PROGRAMMING

ADVANCED PROGRAMMING CONCEPTS - INDIVIDUAL ASSIGNMENT

A COMPARATIVE STUDY OF TRADITIONAL AND AI-ASSISTED PROGRAMMING APPROACHES USING A STUDENT RESULT MANAGEMENT SYSTEM

NAME : L.K.G.Bandara
REGISTRATION NUMBER : 2023s19936
UNIVERSITY OF COLOMBO
FACULTY OF SCIENCE
2026

TASK 1 – CONCEPTUAL UNDERSTANDING

1.1 Introduction to AI-Assisted Programming

AI-assisted programming is a software development approach in which artificial intelligence tools support programmers during the process of designing, writing, debugging, testing, and improving source code. Unlike traditional programming, where the programmer manually develops most of the program logic and searches for solutions independently, AI-assisted programming allows developers to interact with intelligent tools using natural-language instructions or prompts.

Modern AI coding tools are capable of generating code, suggesting solutions, explaining programming concepts, identifying possible errors, and recommending improvements to existing implementations. These tools do not completely replace the programmer. Instead, they act as development assistants that can reduce repetitive work and support decision-making during the software development process.

In AI-assisted programming, the quality of the developer's instructions is important. A clear prompt containing the programming language, system requirements, existing problems, and expected output can produce a more relevant solution. However, the programmer must still examine, test, and understand the generated code before integrating it into a software project.

Therefore, AI-assisted programming can be considered a collaborative approach between the human developer and an AI tool. The developer provides requirements, technical context, and critical judgment, while the AI tool provides suggestions, code generation, explanations, and possible improvements.

1.2 Types of AI Coding Tools

Several categories of AI tools are currently used in software development. One category is code completion tools, which predict and suggest source code while a programmer is typing. These tools can generate individual statements, methods, and larger blocks of code based on the existing programming context.

A second category is conversational AI assistants. These tools allow programmers to describe a problem using natural language and receive code, explanations, debugging guidance, or design suggestions. ChatGPT is an example of this type of tool and was used as the AI assistant for the practical implementation in this assessment.

AI-based debugging and code review tools form another category. Such tools can analyse existing code, identify possible defects or weaknesses, and suggest improvements. They may help developers recognise missing input validation, exception-handling problems, duplicated logic, or maintainability issues.

AI tools can also support software design and documentation. A programmer may use AI to suggest class responsibilities, object-oriented structures, testing scenarios, or documentation content. Therefore, AI-assisted development is not limited to generating source code; it can support several stages of the software development lifecycle.

1.3 Advantages of AI-Assisted Programming

One major advantage of AI-assisted programming is increased development speed. AI tools can quickly generate initial code structures and suggest implementations for common programming requirements. This reduces the amount of time spent writing repetitive code and searching for syntax-related solutions.

AI assistance can also improve debugging. When a programmer clearly describes an error and provides the relevant code or error message, an AI tool can analyse the situation and suggest possible causes and corrections. This is particularly useful during development when errors are caused by constructor mismatches, incorrect variable assignments, input-handling problems, or missing validation.

Another advantage is improved code quality. AI tools can suggest separation of responsibilities and reusable classes. For example, validation logic can be moved from a large main method into a dedicated input validation class. Such improvements can increase readability and maintainability.

AI tools can additionally support learning. Generated code can be accompanied by explanations of exception handling, encapsulation, methods, loops, collections, and object-oriented design. However, this educational benefit is achieved only when the programmer studies and understands the suggested code instead of copying it without analysis.

1.4 Risks and Limitations of AI-Assisted Programming

Although AI-assisted programming provides several benefits, it also introduces risks and limitations. AI-generated code is not guaranteed to be correct. A generated solution may contain logical errors, use unsuitable design decisions, or fail to consider important edge cases. Therefore, all generated code must be reviewed and tested by the programmer.

Another risk is overdependence on AI tools. If students or developers continuously copy generated code without understanding its operation, their problem-solving and debugging skills may become weaker. This is particularly important in programming education, where understanding algorithms, program flow, data structures, and object-oriented concepts is more important than simply obtaining a working program.

Security and privacy are also concerns. Developers should avoid sharing passwords, authentication credentials, private source code, confidential organisational information, or sensitive user data with external AI systems. AI-generated code may also introduce security weaknesses if the developer accepts it without conducting appropriate security reviews.

AI tools may produce solutions that appear professional but are unnecessarily complex or inconsistent with project requirements. For this reason, human judgment remains essential. The programmer is responsible for verifying that generated code is correct, secure, maintainable, and appropriate for the intended application.

TASK 2 – PRACTICAL IMPLEMENTATION

2.1 Selected Programming Problem

Although AI-assisted programming provides several benefits, it also introduces risks and limitations. AI-generated code is not guaranteed to be correct. A generated solution may contain logical errors, use unsuitable design decisions, or fail to consider important edge cases. Therefore, all generated code must be reviewed and tested by the programmer.

Another risk is overdependence on AI tools. If students or developers continuously copy generated code without understanding its operation, their problem-solving and debugging skills may become weaker. This is particularly important in programming education, where understanding algorithms, program flow, data structures, and object-oriented concepts is more important than simply obtaining a working program.

Security and privacy are also concerns. Developers should avoid sharing passwords, authentication credentials, private source code, confidential organisational information, or sensitive user data with external AI systems. AI-generated code may also introduce security weaknesses if the developer accepts it without conducting appropriate security reviews.

AI tools may produce solutions that appear professional but are unnecessarily complex or inconsistent with project requirements. For this reason, human judgment remains essential. The programmer is responsible for verifying that generated code is correct, secure, maintainable, and appropriate for the intended application.

2.2 System Requirements

The Student Result Management System was designed to satisfy the following functional requirements:

1. Add a student using a student ID and student name.
2. Enter marks for Programming, Database Systems, and Mathematics.
3. Calculate the average of the three subject marks.
4. Determine the student's grade based on the calculated average.

5. View the result of an individual student.
6. Search for a student using the student ID.
7. Display all student records in a readable table.
8. Exit the application through the main menu.

The grading criteria used in both implementations were as follows:

Average Mark	Grade
75 and above	A
65-74	B
55-64	C
45-54	D
Below 45	F

In addition to the basic functional requirements, the AI-assisted version was designed to improve the weaknesses identified during the testing of the traditional implementation. These improvements included validation of marks between 0 and 100, safe handling of non-numeric input, prevention of duplicate student IDs, and improved separation of responsibilities between classes.

2.3 System Design and UML Diagrams

Unified Modeling Language (UML) diagrams were used to represent the structure and behaviour of the Student Result Management System. A Use Case Diagram, Class Diagram, and Activity Diagram were developed. These diagrams provide different views of the system and support the understanding of its requirements, object-oriented structure, and operational workflow.

2.3.1 Use Case Diagram

The Use Case Diagram represents the interactions between the user and the Student Result Management System. The main user can add students, enter student marks, view student results, search for students, and display all student records. The calculation of the average and determination of the grade are internal system functions performed as part of result processing.

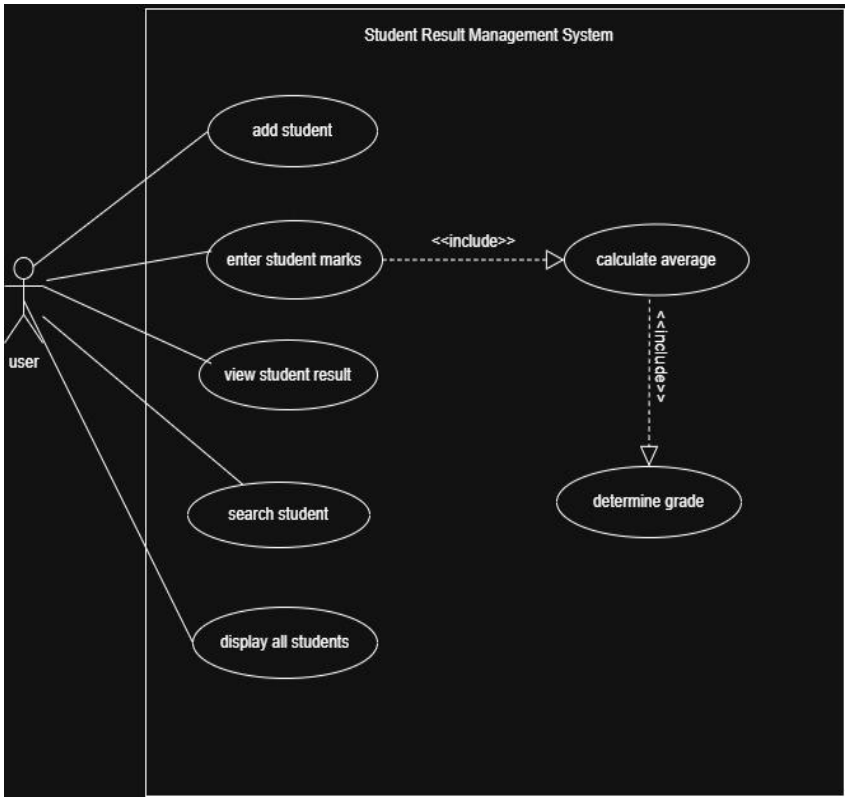

Figure 1: Use Case Diagram of the Student Result Management System

2.3.2 Class Diagram

The Class Diagram represents the object-oriented structure of the AI-assisted implementation. The final system consists of four main classes: Student, StudentManager, InputValidator, and Main.

The Student class encapsulates individual student information, subject marks, the calculated average, and the grade. StudentManager is responsible for storing and searching student objects and preventing duplicate student IDs. InputValidator contains reusable methods for validating integer values, marks, and non-empty string inputs. The Main class coordinates the menu and the overall application flow.

The separation of these responsibilities improves the readability and maintainability of the AI-assisted implementation.

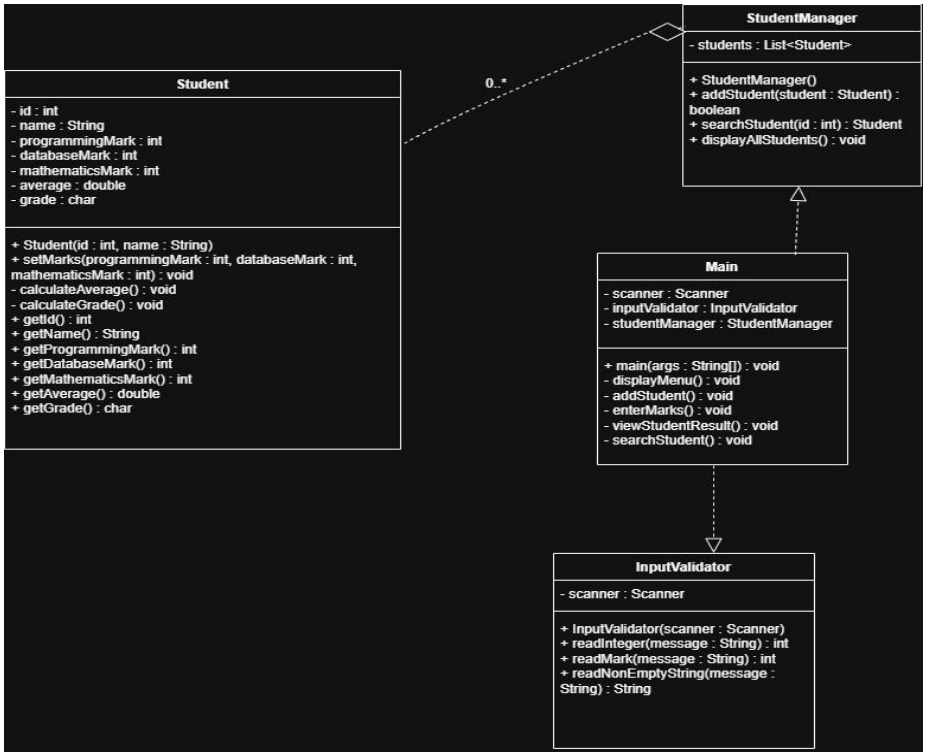

Figure 2: Class Diagram of the AI-Assisted Student Result Management System

2.3.3 Activity Diagram

The Activity Diagram illustrates the overall workflow of the Student Result Management System. The application begins by displaying the main menu and reading the user's choice. The selected operation is then performed according to the menu option.

Decision points are used for input validation, duplicate student ID checking, student searching, and mark validation. After completing an operation, the application returns to the main menu. The system terminates only when the user selects the Exit option. This activity flow reflects the improved behaviour of the AI-assisted implementation.

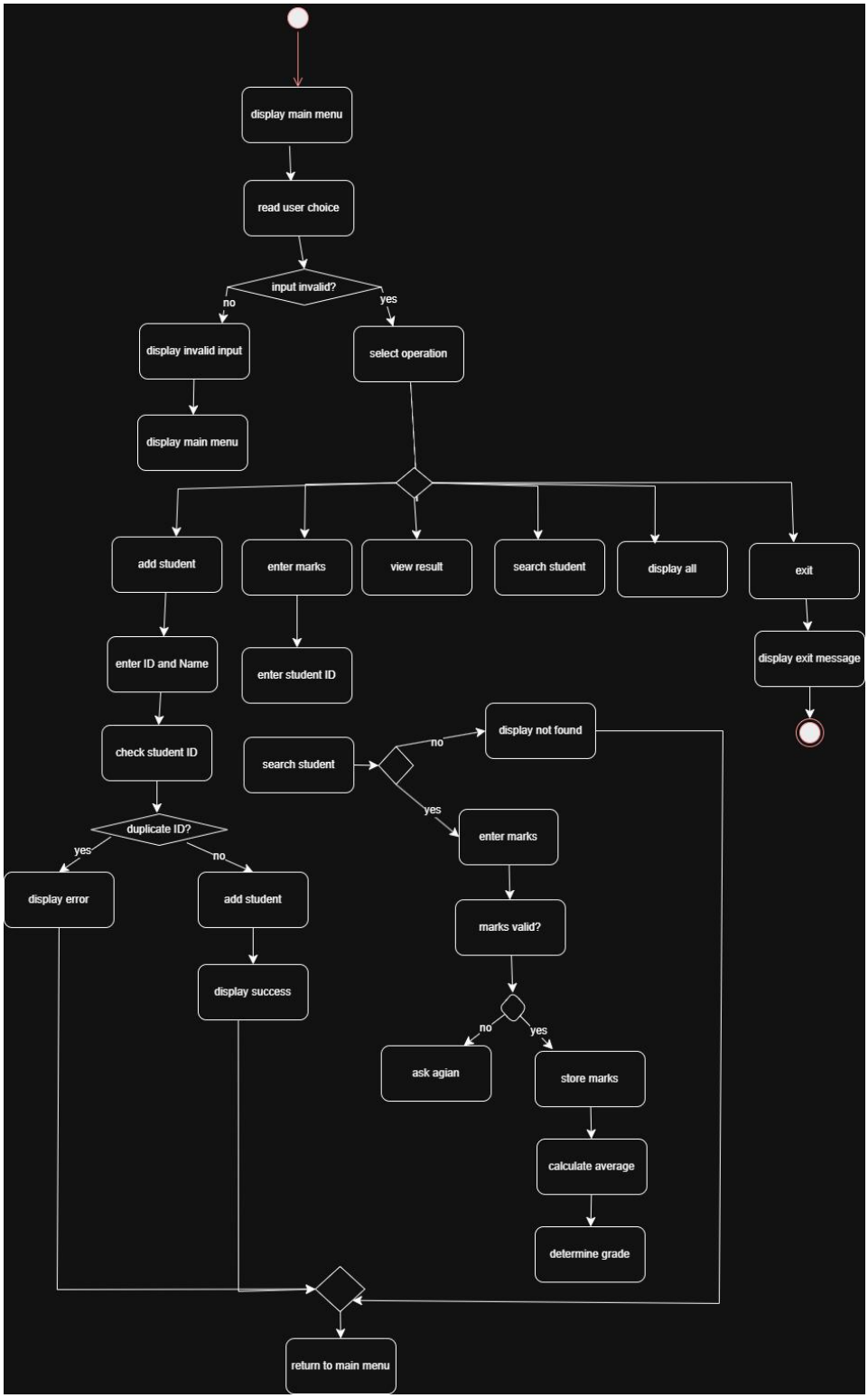

Figure 3: Activity Diagram of the Student Result Management System

2.4 Version A – Traditional Programming Approach

Version A of the Student Result Management System was developed using a traditional programming approach. The implementation consisted primarily of the

Student, StudentManager, and Main classes. The program logic was manually constructed by defining the student data model, implementing result calculations, managing an ArrayList of Student objects, and creating a console-based menu.

The Student class stored the student ID, name, subject marks, average, and grade. Separate methods were initially used to set marks, calculate the average, and calculate the grade. StudentManager maintained the collection of student objects and provided methods to add, search, and display students. The Main class directly used the Scanner class to read user input and coordinate menu operations.

During traditional development, several programming problems were encountered. A constructor argument mismatch occurred because the Student constructor initially required more parameters than were supplied when creating a Student object. In another test, the student name was displayed as null because the name field was not correctly assigned in the constructor. These problems were manually identified and corrected during development.

After completing the basic implementation, the system successfully added students, recorded marks, calculated averages and grades, searched for students, and displayed all student records.

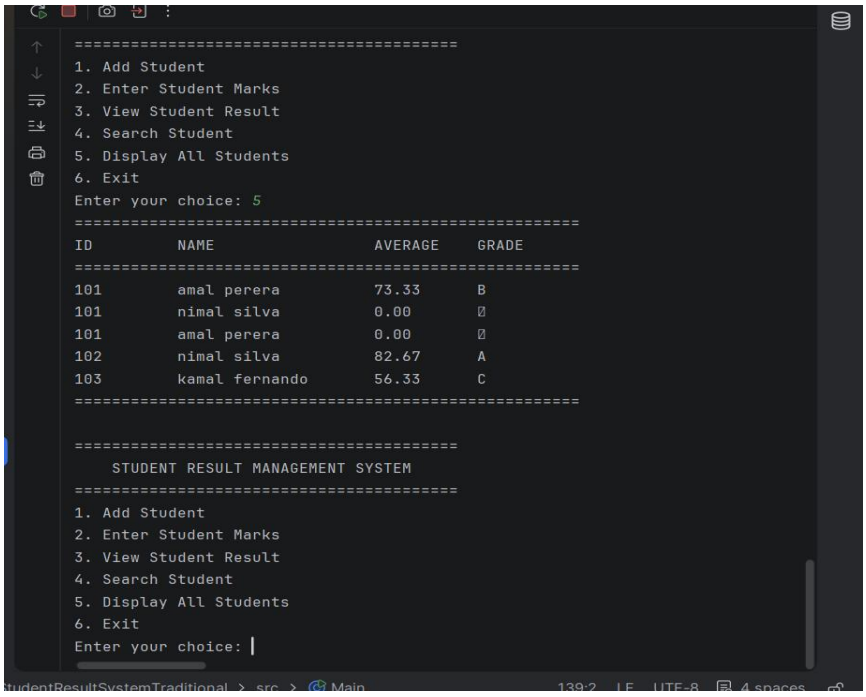

Figure 4: Successful execution of Version A using the traditional programming approach

Although the normal execution was successful, further testing identified weaknesses in the traditional implementation. Marks greater than 100 and negative marks were accepted by the system. A non-numeric menu input caused an InputMismatchException and terminated the application. In addition, multiple students could be added using the same student ID. These weaknesses demonstrated the need for stronger input validation and error handling.

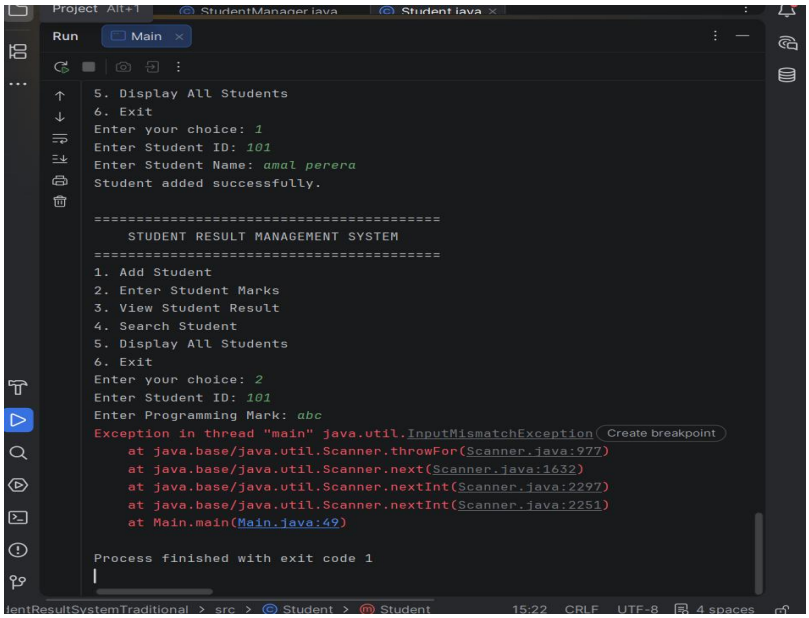

Figure 5: InputMismatchException caused by non-numeric input in Version A

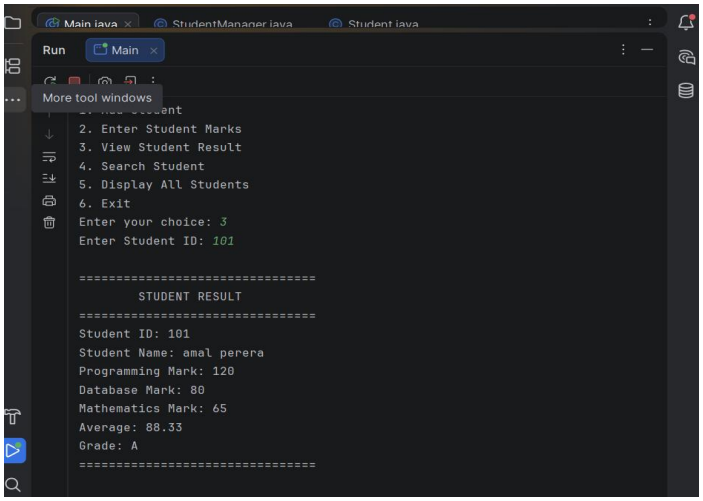

Figure 6: Acceptance of an invalid mark greater than 100 in Version A

2.5 Version B – AI-Assisted Programming Approach

Version B was developed with assistance from ChatGPT. The weaknesses identified in Version A were provided to the AI tool as part of the development context. Instead of requesting a complete program using a single prompt, AI assistance was applied

incrementally to system design, Student class improvement, input validation, duplicate student ID handling, and Main class integration.

The AI-assisted design introduced a separate InputValidator class. This class contains reusable methods for reading integer values, validating marks, and reading non-empty string inputs. Non-numeric input is first read as a String and converted using Integer.parseInt(). A NumberFormatException is caught when conversion fails, allowing the program to display an error message and request another input instead of terminating.

The readMark() method restricts marks to the range from 0 to 100. If a mark is outside this range, the value is rejected and the user is asked to enter the mark again. This directly addresses the invalid mark and negative mark weaknesses found in Version A.

The StudentManager class was also improved to prevent duplicate student IDs. Before adding a new Student object, the system searches the existing collection for the specified ID. The addStudent() method returns a boolean value to indicate whether the student was successfully added.

The Student class was refined so that the average and grade are automatically calculated when subject marks are set. This reduces the risk of another class setting marks but forgetting to call the calculation methods. The calculation methods were made private because they represent internal Student behaviour.

Finally, the Main class was divided into separate methods such as addStudent(), enterMarks(), viewStudentResult(), and searchStudent(). This improved readability by reducing the amount of operational logic directly inside the main method.

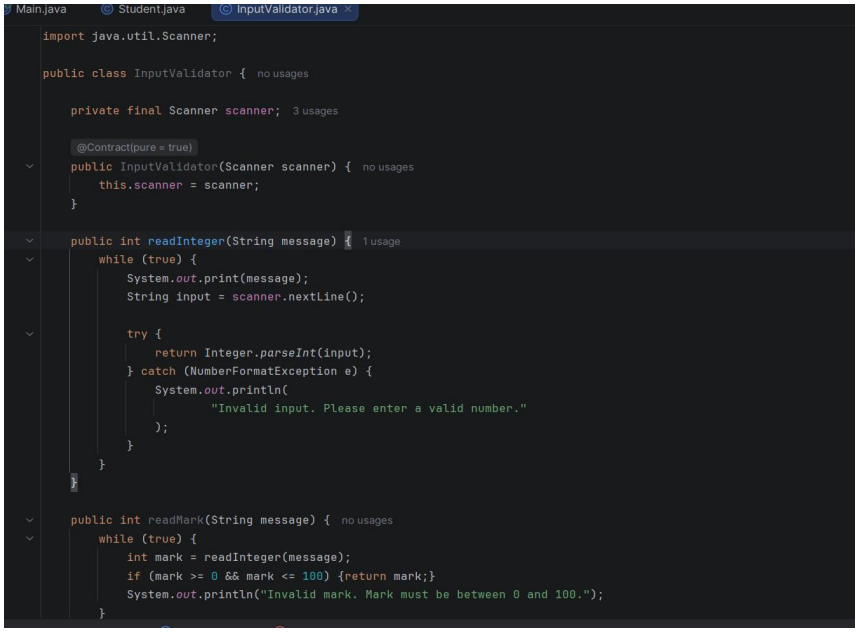

Figure 7: AI-assisted InputValidator class used for reusable input validation

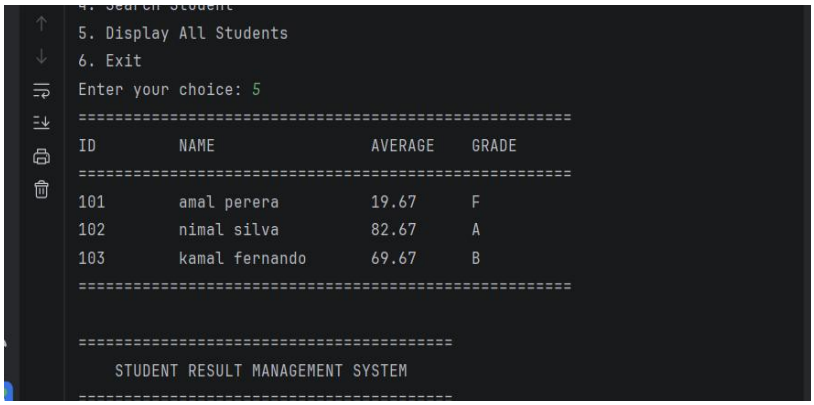

Figure 8: Successful execution of the AI-assisted Version B

2.6 AI Prompts Used During Development

ChatGPT was used as the AI coding assistant during the development of Version B. The prompts were designed to provide the system requirements and specific weaknesses identified during traditional implementation. The main prompts used are summarised below. The complete prompt record and screenshots are included in the GitHub repository.

prompt	purpose
Prompt1	Analyse traditional implementation weaknesses and suggest an improved object-oriented design
Prompt2	Generate and improve the Student class

Prompt3	Create reusable input validation and exception handling
Prompt4	Prevent duplicate student IDs using an improved StudentManager
Prompt5	Integrate the classes through a readable Main class

One of the most important prompts described the errors observed in Version A, including the InputMismatchException, acceptance of marks greater than 100, and acceptance of negative marks. ChatGPT was asked to create a reusable InputValidator class that safely reads integer input, validates marks, and requests another value when invalid input is entered.

The AI-generated suggestions were reviewed and integrated into the Version B project. AI-assisted code can be identified particularly in the InputValidator class, the duplicate ID checking logic in StudentManager, the automatic result calculation design in Student, and the refactored menu operation methods in Main.

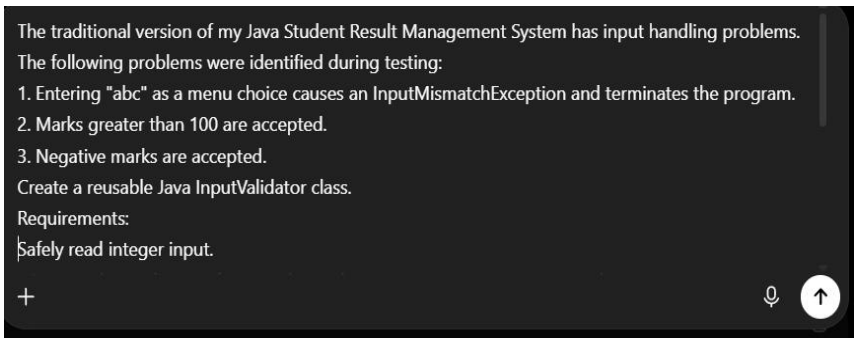

Figure 9: Example prompt used to request AI-assisted input validation and exception handling

2.7 Testing and Error Handling

The same major test scenarios were applied to Version A and Version B to compare their error-handling behaviour. The tests included normal student data, a mark greater than 100, a negative mark, non-numeric menu input, and a duplicate student ID.

Test Scenario	Version A result	Version B result
Normal student data	Successfully processed	Successfully processed
Mark = 120	Accepted invalid mark	Rejected and requested another mark

Mark = -20	Accepted negative mark	Rejected and requested another mark
Menu input = abc	Application terminated with InputMismatchException	Displayed an error and requested input again
Duplicate ID = 101	Duplicate student accepted	Duplicate student rejected

The test results show a clear improvement in error handling in Version B. The AI-assisted implementation did not terminate when non-numeric input was entered. Instead, the exception was handled and the input operation continued. Invalid marks were rejected through range validation, and duplicate IDs were prevented before student records were added.

Therefore, the AI-assisted implementation demonstrated greater robustness under invalid input conditions.

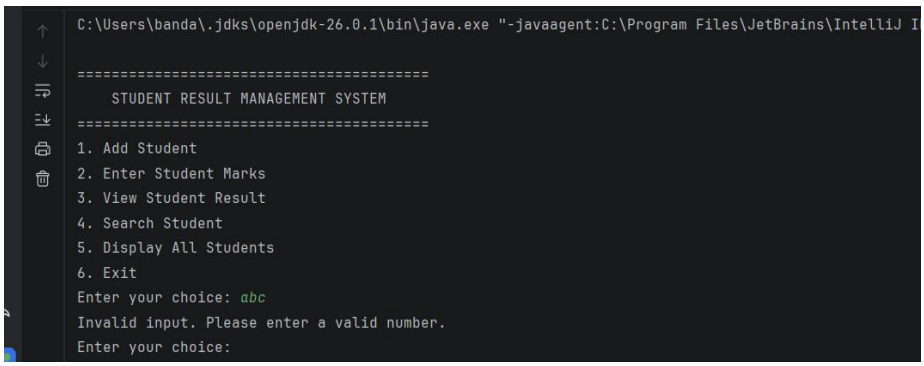

Figure 10: Non-numeric input handled without application termination in Version B

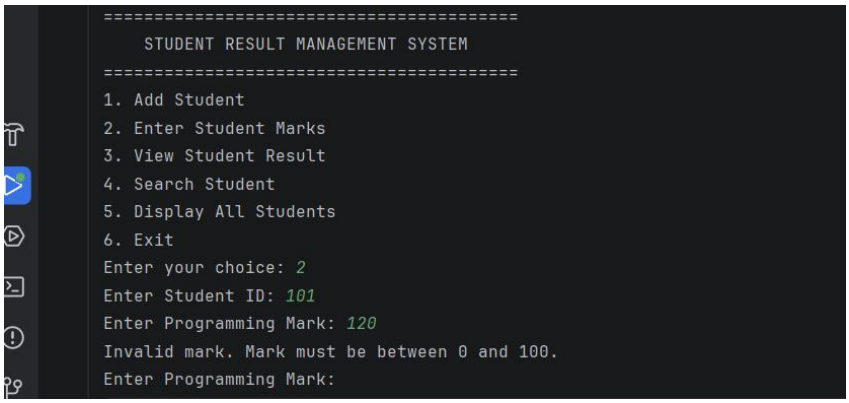

Figure 12: Duplicate student ID rejected in Version B

2.8 Comparison of Traditional and AI-Assisted Programming

Comparision factor	Version A	Version B
Development time	29 hours 47 minutes	31 hours 29 minutes
Class structure	Student, StudentManager, Main	Student, StudentManager, InputValidator, Main
Input handling	Direct Scanner input	Reusable InputValidator
Non-numeric input	Program terminated	Exception handled
Mark validation	No range validation	0-100 validation
Duplicate IDs	accepted	rejected
Result calculation	Separate calculation method calls	Automatic after setting marks
readability	More logic in Main	Menu operations separated into methods
maintainability	Validation changes require manual modification	Validation logic centralised and reusable

The recorded elapsed development period for Version A was 29 hours and 47 minutes, while the recorded elapsed period for Version B was 31 hours and 29 minutes. These values represent the periods between the recorded start and end timestamps and include breaks between development sessions. Therefore, they should not be interpreted as continuous active coding time.

Although Version B had a longer recorded elapsed period, the AI-assisted development process also included preparing and refining prompts, reviewing AI-generated suggestions, understanding the proposed code, integrating separate classes, conducting invalid-input tests, capturing evidence, and documenting the AI-assisted process. Therefore, development speed cannot be evaluated using the elapsed period alone.

The main advantage observed in Version B was not simply a reduction in recorded time. Instead, AI assistance helped identify and implement improvements related to input validation, exception handling, duplicate ID prevention, readability, and separation of responsibilities. The comparison suggests that AI assistance was

particularly useful for improving the quality and robustness of the implementation after weaknesses had been identified in the traditional version.

TASK 3 – CRITICAL ANALYSIS

3.1 How AI Changed My Coding Approach

The development of the two versions demonstrated a noticeable difference between my traditional and AI-assisted programming approaches. During the development of Version A, I mainly focused on implementing the functional requirements of the Student Result Management System. I manually designed the classes, wrote the program logic, and corrected compilation and runtime problems as they appeared. My main objective was initially to make the required functions work correctly.

This traditional approach required more manual debugging and experimentation. For example, I encountered a constructor argument mismatch when creating Student objects. I also experienced a problem where the student name was displayed as null because the constructor did not correctly assign the name field. These problems required me to examine the relationship between constructor parameters, object fields, and object creation statements before correcting the implementation.

However, the testing stage showed that my initial programming approach concentrated mainly on normal execution. Although the application could successfully add students, enter marks, calculate results, and display student information, I had not initially considered several invalid input conditions. The system accepted a mark of 120, accepted negative marks, terminated when non-numeric menu input was entered, and allowed duplicate student IDs.

In Version B, my coding approach became more problem-oriented and iterative. Instead of only asking the AI tool to generate a complete application, I used the weaknesses identified in Version A as input for separate prompts. I requested improvements for system structure, input validation, exception handling, duplicate ID checking, and integration of the classes.

AI assistance also encouraged me to think about separation of responsibilities. In Version A, input operations were handled directly using Scanner in the Main class. In Version B, the InputValidator class centralised the validation logic. The Main class was also divided into separate methods for different menu operations. Therefore, AI changed my approach from primarily focusing on whether the program worked under normal conditions to considering robustness, edge cases, readability, and maintainability.

The development time also showed a difference between the two approaches. Version A required 32 hours and 47 minutes, while Version B required 29 hours and 31 minutes. Therefore, the AI-assisted version required 3 hours and 16 minutes less development time based on my recorded development time. However, the main benefit was not only the time reduction. AI assistance also provided targeted suggestions for weaknesses that had already been identified through testing.

3.2 Impact of AI on My Understanding of Programming Concepts

In my experience, AI assistance improved my understanding of several programming concepts when I actively examined the suggested code. One important example was exception handling. In Version A, Scanner was used directly to read integer values. When a non-numeric value such as **abc** was entered for a menu option, the application terminated with an **InputMismatchException**.

In the AI-assisted implementation, input was read as a String and converted to an integer using Integer.parseInt(). The conversion was placed inside a try-catch structure, and NumberFormatException was handled when the entered text could not be converted to an integer. By testing this behaviour, I gained a clearer understanding of how exceptions can be handled without terminating the entire application.

The AI-assisted implementation also improved my understanding of separation of responsibilities in object-oriented programming. Previously, I understood that a program could be divided into classes, but the **InputValidator** design provided a practical example of why a separate class can be useful. Instead of repeating

validation logic in different methods of Main, the same validation methods could be reused.

Another useful concept was encapsulation of object behaviour. In the improved Student class, setting the marks automatically triggers the calculation of the average and grade. The methods responsible for these calculations are private because they are internal operations of the Student object. This design reduced the possibility of another class setting marks but forgetting to calculate the result.

However, AI assistance can weaken understanding if generated code is copied without examination. When AI provides a working solution quickly, there is a temptation to accept the code without understanding why particular methods, exceptions, or class structures are used. During this assessment, I had to read the suggested code, integrate it into IntelliJ IDEA, resolve class and constructor relationships, and test the behaviour using different inputs.

Therefore, I believe AI improved my understanding when it was used as a learning and development assistant. The improvement depended on reviewing the generated code and relating it to the errors observed in my own implementation. If I had only copied the final code without testing or analysing it, the same AI assistance could have weakened my programming understanding.

3.3 Situations Where AI Should Not Be Used

AI coding tools should not be used without human supervision in situations where software failure may create serious consequences. Examples include safety-critical medical systems, aircraft control systems, industrial safety systems, and other applications where incorrect software behaviour may cause injury or loss of life. AI may support developers in these areas, but generated code should not be accepted without formal verification, specialised testing, and expert review.

AI should also not be trusted independently for security-critical programming. Authentication systems, encryption-related implementations, access control mechanisms, and systems that process sensitive data require careful security analysis.

AI-generated code may appear correct while still containing vulnerabilities or inappropriate security practices.

Another situation where AI should not be used is when the developer cannot understand or verify the generated solution. A programmer remains responsible for the behaviour of the software being developed. If the developer is unable to explain the code, identify its assumptions, or test its behaviour, integrating the generated code introduces unnecessary risk.

In programming education, AI should not be used to bypass the learning process or violate assessment requirements. Generating an entire solution and submitting it as independently written work can create academic integrity problems and may prevent the student from developing essential programming and problem-solving skills. AI use should follow the instructions and policies of the relevant course or assessment.

AI should also not be provided with confidential information, passwords, API keys, private user information, or sensitive organisational source code unless the selected AI service and organisational policies specifically permit such use. Developers should carefully consider privacy and confidentiality before sharing programming context with an external AI system.

3.4 Ethical Considerations of AI-Generated Code

The increasing use of AI-generated code introduces several ethical considerations. One important issue is responsibility. Although an AI tool may generate or suggest source code, the developer who integrates the code into an application remains responsible for reviewing and testing its behaviour. Blaming the AI tool for a software defect does not remove the developer's responsibility to verify the implementation.

Transparency is another important consideration. When AI assistance is a significant part of an academic or professional development process, its use should be disclosed where required. In this assessment, Version B was explicitly identified as the AI-assisted implementation, ChatGPT was identified as the AI tool, and the prompts used

during development were recorded. This provides evidence of how AI contributed to the project.

Intellectual property and licensing should also be considered. Developers should not automatically assume that every generated code suggestion is appropriate for unrestricted use. When AI-generated code resembles an external library or implementation, the developer may need to examine licensing requirements and the origin of dependencies before using the code in a commercial system.

Bias and unequal access to AI tools are additional ethical concerns. Different developers may have access to AI systems with different capabilities, which can influence productivity and learning opportunities. AI-generated solutions may also reflect limitations or biases in the data and systems used to develop the AI model.

Academic integrity is particularly important for students. AI should be used according to assessment rules and should not be presented as independent human work when disclosure is required. In this assessment, AI assistance was intentionally used only for Version B as part of the comparison between traditional and AI-assisted programming.

Finally, developers should consider privacy when interacting with AI coding assistants. Sensitive user data, credentials, private keys, and confidential source code should not be included in prompts without appropriate permission and protection. Ethical AI-assisted programming therefore requires transparency, human responsibility, critical review, and careful handling of data.

TASK 4 – CONCLUSION

This assessment compared traditional programming and AI-assisted programming through the development of two versions of a Java console-based Student Result Management System. Both implementations were developed using the same basic functional requirements, including adding students, entering subject marks, calculating averages and grades, searching for student records, and displaying results.

Version A was developed using a traditional programming approach without AI assistance. The implementation successfully achieved the basic functional requirements, but the development and testing process revealed several weaknesses. Marks greater than 100 and negative marks were accepted, non-numeric menu input caused an InputMismatchException, and duplicate student IDs were allowed. The traditional development process required 32 hours and 47 minutes based on the recorded development time.

Version B was developed with assistance from ChatGPT. The errors and weaknesses identified in Version A were used to create more specific prompts for the AI tool. The AI-assisted implementation introduced a reusable InputValidator class, NumberFormatException handling, mark range validation, duplicate ID prevention, automatic calculation of averages and grades, and improved separation of responsibilities between classes.

The AI-assisted version required 29 hours and 31 minutes of recorded development time, which was 3 hours and 16 minutes less than Version A. In addition to this reduction in development time, Version B demonstrated better robustness, readability, and maintainability. Testing showed that invalid marks were rejected, non-numeric input was handled without application termination, and duplicate student IDs were prevented.

The assessment also demonstrated that AI-generated code cannot be accepted without human review. The developer must understand the generated suggestions, verify that they satisfy the requirements, integrate them correctly, and test both normal and invalid input scenarios. AI assistance was most effective when specific problems and observed weaknesses were clearly described in the prompts.

From this practical comparison, I conclude that AI-assisted programming can improve software development efficiency and code quality when it is used responsibly. However, AI should be treated as an assistant rather than a replacement for programming knowledge and human judgment. The programmer remains responsible for understanding, testing, and maintaining the final software system.

GITHUB REPOSITORY

The complete source code for Version A and Version B, AI prompts, screenshots, UML diagrams, development log, and project documentation are available in the GitHub repository provided below.

https://github.com/GeethmaBandara/AI-Assisted-Programming-Assessment